

Python on the IBMi- Chapter 5



With the web server behind him and the shadows of the cave growing distant, our hero now faces a new challenge: a wide, rushing river separating the front-end world of Python from the powerful domain of RPG. There's no grand bridge or ferry, only logs, rope, and grit. And so, he builds a makeshift raft and sets sail.

In this chapter, we begin tying it all together. We'll create a Product Master table filled with sample data, then construct a Python web server that can handle incoming requests. When a traveler knocks at our digital door with a GET request, the server will forward that message to a backend RPG program. The RPG logic, steady and reliable, will respond with a list of products in clean, structured JSON.

This crossing isn't without its currents but with a steady hand and a clear goal, the raft holds strong.

So grab your rope, your keyboard, and your courage, it's time to cross the river.

Disclaimer

This article will use the built-in Python HTTP Server. While this works, it is NOT production ready. While it might work for simple utilities and very low demand. It's a very simple HTTP Server that handles basic requests. The next few articles are for demonstration purposes only.

Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)

Objective

- Create a sample Product Master Table and populate it with data.
- Write a Python script to act as a web server
- Create a backend RPG program to take requests from the web server and return data.

Before We Begin

There's going to be a lot of information in this article. And in this limited space I can't explain every line of the code hopefully it self-documenting enough that you can follow.

The table and RPG program I put in RTHOMPSON1. Please change any references you need to change to make this work on your system.

On the Python Web Server code, I convert the Request Data to JSON to make it easier to use in RPG. In RPG we don't have a great way of handling URL Query Strings. A query string will look something like (**/api?app=product&action=getproductlist**). There are a lot of other things that can happen to a URL like it can have characters escaped to handle special characters. Converting the data to JSON just makes dealing with it in RPG easier. And it's also very easy to convert the data in Python. While I wish Scott Forstie would give us a SQL Service that would handle all the URL scenarios we aren't there yet.

Let's Code

First let's create a test database table with some test data. This will create an example Product Master table with some sample data.

product_master.sql

```
CREATE OR REPLACE TABLE rthompson1.product_master (  
    product_number VARCHAR(20) PRIMARY KEY,  
    description VARCHAR(100),  
    cost DECIMAL(10, 2),  
    unit_of_measure VARCHAR(10),  
    category VARCHAR(50)  
)  
On Replace Delete Rows;
```

```

INSERT INTO rthompson1.product_master (product_number,
description, cost, unit_of_measure, category) VALUES
('P0001', 'Wireless Mouse', 19.99, 'EACH', 'Electronics'),
('P0002', 'Bluetooth Keyboard', 29.95, 'EACH', 'Electronics'),
('P0003', 'USB-C Charger', 14.50, 'EACH', 'Electronics'),
('P0004', 'Laptop Backpack', 45.00, 'EACH', 'Accessories'),
('P0005', '16GB USB Drive', 9.99, 'EACH', 'Electronics'),
('P0006', 'Coffee Mug', 5.25, 'EACH', 'Kitchen'),
('P0007', 'Notebook - Lined', 3.15, 'EACH', 'Stationery'),
('P0008', 'Ballpoint Pens (Pack of 10)', 2.99, 'PACK',
'Stationery'),
('P0009', 'Desk Lamp', 18.75, 'EACH', 'Office'),
('P0010', 'Wireless Headphones', 59.95, 'EACH', 'Electronics'),
('P0011', 'Stapler', 6.49, 'EACH', 'Office'),
('P0012', 'Pack of Sticky Notes', 1.99, 'PACK', 'Stationery'),
('P0013', 'AA Batteries (4 Pack)', 4.25, 'PACK', 'Electronics'),
('P0014', 'Toaster Oven', 39.95, 'EACH', 'Kitchen'),
('P0015', 'Water Bottle 1L', 7.89, 'EACH', 'Kitchen'),
('P0016', 'LED Light Bulbs (6 Pack)', 12.49, 'PACK', 'Home
Improvement'),
('P0017', 'Phone Charger Cable', 8.95, 'EACH', 'Electronics'),
('P0018', 'Notebook Cooling Pad', 21.99, 'EACH', 'Electronics'),
('P0019', 'Mechanical Pencil Set (2 Pack)', 4.75, 'PACK',
'Stationery'),
('P0020', 'Electric Kettle', 24.50, 'EACH', 'Kitchen');

```

Build an RPG Program to handle the web requests

We are going to create an RPG program that will handle HTTP Get Requests from our Python web server. When the Web Server gets the request if it's an API call it will call the RPG program to get the data and return it to the web server.

The program will have 3 parameters.

- Method - which will be GET or POST from the web server
- Request Data - A JSON String that will contain the request data. (ie {"app": "PRODUCT", "action": "GETPRODUCTLIST"}) this will tell the RPG program what you are requesting.
- Return Data - The JSON data we are returning to the Python Script.

I called the SQLRPGLE Program WEBHNDLR.SQLRPGLE. It will take the incoming parameter, parse the JSON request data and call the sub procedure to return a JSON string back to the web server as an output parameter pResponseData.

webhndlr.sqlrpgle

```
**free
Ctl-Opt option(*nodebugio:*srcstmt:*nounref) Main(Main)
dftactgrp(*no)
    text('Web Server RPG Handler Program');

// The main procedure that will listen for incoming requests
Dcl-Proc Main;
    Dcl-Pi *N;
        pMethod CHAR(10) const;
        pRequestData CHAR(32000) const;
        pResponseData CHAR(32000);
    End-Pi;

    Dcl-S vJsonString CHAR(32000);
    Dcl-S vApp VARCHAR(50);
    Dcl-S vAction VARCHAR(50);
    // Dcl-S vRequestData SQLTYPE(CLOB:500000);

    Exec Sql
        Set Option commit = *none;

    If %upper(pMethod) = 'GET ';

        // If Request Data is blank return null
        If pRequestData = '';
            pResponseData = '{"data": null}';
            Return;
        Else;
            Exec Sql
                SELECT upper(jt.app)
                    ,upper(jt.action)
                INTO :vApp
                    ,:vAction
                FROM JSON_TABLE(trim(:pRequestData),
                    'lax $'
                    COLUMNS (
                        app VARCHAR(50) PATH 'lax $.app',
                        action VARCHAR(50) PATH 'lax $.action'
                    )
                ) AS jt;

            If sqlcod <> 0;
                pResponseData = '{ "error": "Invalid JSON format" }';
                Return;
            EndIf;
        EndIf;
    End-Proc;
```

```

        EndIf;

        Select;
            When vApp = 'PRODUCT' and vAction = 'GETPRODUCTLIST';
                // Call the procedure to get the Product List in JSON
format
                vJsonString = GetProductList();
                pResponseData = vJsonString;
            Other;
                pResponseData = '{ "error": "Unknown action" }';
            EndSl;
        EndIf;
    End-Proc;

Dcl-Proc GetProductList;
    Dcl-Pi *N CHAR(32000);
    End-Pi;

    Dcl-S jsonString CHAR(32000);
    Dcl-S jsonStringClob SQLTYPE(CLOB:32000);

    Exec Sql
        Select Cast(
            Json_Object(
                'data' Value (Json_Arrayagg(
                    Json_Object(
                        'ProductNumber' Value rtrim(product_number)
                        , 'Description' Value Rtrim(description)
                        , 'Cost' Value Cost
                        , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
                        , 'Category' Value Rtrim(category)
                    )
                )
            ) As Clob(32000))
        Into :jsonstringclob
        From rthompson1.product_master;

    JsonString = JsonStringClob_Data;
    Return JsonString;
End-Proc;

```

Don't forget to compile the RPG Program.

index.html

We will create a basic index.html in our folder (**/home/rthompson/python/chapter5**). This (**index.html**) will have two buttons on the screen. One button to get the product list in JSON format. And the other to shutdown the Python server without having to kill it from PASE.

Get Product List

Shutdown

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>API Actions</title>
</head>
<body>
  <div class="button-container">
    <button
onclick="window.location.href='/api?app=product&action=getproduc
tlist'">Get Product List</button>
    <br><br>
    <button
onclick="window.location.href='/api?action=shutdown'">Shutdown</
button>
  </div>
</body>
</html>
```

Create the Python Script to handle the HTTP Requests and call the RPG programs

Using the same folder (**/home/rthompson/python/chapter5**) we are going to create a new Python script I called (**http_server.py**). I can't go into all the details of the Python Script in this article but here are the highlights.

The script will accept two parameters when starting. One for the port number you want to use and one for the library you placed your RPG program in. The script will start the web server using the port or 43001 as the default. It will wait for incoming requests. If the URL has /api the script knows this means we want to call the RPG program. If it doesn't have /api in the URL it

will try to open either the file you specify or index.html if no file is specified. So imagine your URL is (<http://myibmi:43001>) it will open the index.html file.

Also, I built a way to shutdown the server. Your URL will look like the following (<http://myibmi:43001/api?action=shutdown>). Python will gracefully end the web server.

I provided the index.html to make it easier to just click a button to get the product list or to shutdown the server.

To get the product list you could manually go to (<http://myibmi:43001/api?app=product&action=getproductlist>) and the web browser should display the product master in JSON format.

If the URL contains /api it will call our RPG program WEBHNDLR.SQLRPGLE and pass along the information.

Below is the Python Script.

[http_server.py](#)

```
# Import the built-in HTTP server class for handling web
requests
from http.server import HTTPServer, SimpleHTTPRequestHandler,
ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

# Import argparse for command-line argument parsing
import argparse
# Import standard libraries for handling JSON
import json
import threading

# Import IBM DB2 connection tools and toolkit
import ibm_db
from toolkit import *
from toolkit.transport import DirectTransport

# Create a transport object that allows the toolkit to talk to
IBM i using SQL
itransport = DirectTransport()
print("Starting HTTP Server... Please wait...")

# -----
# DEFINE HTTP REQUEST HANDLER CLASS
# -----

class RPGHTTPRequestHandler(SimpleHTTPRequestHandler):
```

```

LIBRARY = 'RTHOMPSON1' # default; will be overridden

"""
Custom HTTP handler that serves static files and also
handles /api GET and POST requests,
calling an RPGLE program using itoolkit.
"""

def call_rpg(self, method, req_data):

    """
    Call the RPGLE program 'WEBSERV' in library,
    passing in the HTTP method and request data as
parameters,
    and returning the RPG response.

    Parameters:
        method (str): 'GET' or 'POST'
        req_data (str): JSON or text from the HTTP request

    Returns:
        str: response string returned from RPG program
    """

    # Initialize the toolkit object
    itool = iToolkit()

    # Add the library to the library list so the program can
be found
    itool.add(iCmd('addlible', f'ADDLIB {self.LIBRARY}'))

    # Add the RPG program call with 3 parameters:
    # - method: 10-character input
    # - request: 32000-character input
    # - response: 32000-character output (blank string
placeholder)
    itool.add(iPgm('pgmcall', 'WEBHNDLR')
                .addParm(iData('method', '10a', method))
                .addParm(iData('request', '32000a', req_data))
                .addParm(iData('response', '32000a', ' '))
                )

    # Execute the RPG call
    itool.call(itransport)

    # Retrieve the output from the RPG program
    result = itool.dict_out('pgmcall')

```



```

        # Return the response value or fallback if missing
        return result.get('response', 'No response')

    # Handle HTTP GET requests to /api by calling the RPG
program
    def do_GET(self):
        if self.path.startswith('/api'):

            # Parse query string into JSON string
            query = urlparse(self.path).query
            query_dict = parse_qs(query)

            # Check for shutdown command
            if query_dict.get('action', [''])[0].upper() ==
'SHUTDOWN':

                print("⚠ Shutdown requested via HTTP")

                def stop_server():
                    print("▼ Shutting down server...")
                    self.server.shutdown()

                threading.Thread(target=stop_server).start()

                self.send_response(200)
                self.send_header('Content-Type',
'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"status": "Server
shutting down"}).encode())
                return

            #Normal GET request processing
            query_json = json.dumps({k: v[0] for k, v in
query_dict.items()})

            request_data = query_json
            print("=== Incoming GET request ===")
            print(f"Query string: {query_json}")

            try:

                # Call the RPG program with method = 'GET'
                rpg_result = self.call_rpg('GET', request_data)
                print(f"Return Data: {rpg_result}")

            # Return the result to the client

```

```

        self.send_response(200)
        self.send_header('Content-Type',
'application/json')
        self.end_headers()
        self.wfile.write(rpg_result.encode())

    except Exception as e:
        print("=== Incoming GET request ===")
        print(f"Query string: {query_json}")

        print("=== RPG Response (raw) ===")
        print(rpg_result)

        print("ERROR during RPG call:", e)
        self.send_response(500)
        self.end_headers()
        self.wfile.write(json.dumps({"error":
str(e)}).encode())
    else:
        # Serve regular static files (like index.html, etc.)
        super().do_GET()

# -----
# START THE SERVER
# -----

if __name__ == '__main__':
    # Setup argument parser
    parser = argparse.ArgumentParser(description='Simple RPG
HTTP Server')
    parser.add_argument('--port', type=int, default=43001,
help='Port to listen on')
    parser.add_argument('--library', type=str,
default='RTHOMPSON1', help='Library to add to library list')

    args = parser.parse_args()

    # Save the library name globally
    RPGHTTPRequestHandler.LIBRARY = args.library

    # Start the HTTP server
    port = args.port
    server = ThreadingHTTPServer(('0.0.0.0', port),
RPGHTTPRequestHandler)
    print(f"Serving on http://localhost:{port} using library
{args.library}")
    print("Press Ctrl+C to stop the server")

```

```

try:
    server.serve_forever()
except KeyboardInterrupt:
    print("\n ⚠ Shutting down server gracefully...")
    server.shutdown()
    server.server_close()
    print(" 🚩 Server shutdown complete")

```

Now let's run the script

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```
cd python/chapter5
```

```
python3 http_server.py --port 43001 --library RTHOMPSON1
```

You should see the server starting. Once it's ready you will see the Serving message.

```

-bash-5.2$ python3 http_server.py --port 43001 --library RTHOMPSON1
Starting HTTP Server... Please wait...
Serving on http://localhost:43001 using library RTHOMPSON1

```

Now navigate to your IBMi in a web browser (<http://myibmi:43001>). If everything is working correctly you should see our index.html file with two buttons. If you click the Get Product List it should return the following data.

```

{"data":[{"ProductNumber":"P0001","Description":"Wireless Mouse","Cost":19.99,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0002","Description":"Bluetooth Keyboard","Cost":29.95,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0003","Description":"USB-C Charger","Cost":14.50,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0004","Description":"Laptop Backpack","Cost":45.00,"UnitOfMeasure":"EACH","Category":"Accessories"},{"ProductNumber":"P0005","Description":"16GB USB Drive","Cost":9.99,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0006","Description":"Coffee Mug","Cost":5.25,"UnitOfMeasure":"EACH","Category":"Kitchen"},{"ProductNumber":"P0007","Description":"Notebook - lined","Cost":3.15,"UnitOfMeasure":"EACH","Category":"Stationery"},{"ProductNumber":"P0008","Description":"Ballpoint Pens (Pack of 10)","Cost":2.99,"UnitOfMeasure":"PACK","Category":"Stationery"},{"ProductNumber":"P0009","Description":"Desk Lamp","Cost":18.75,"UnitOfMeasure":"EACH","Category":"Office"},{"ProductNumber":"P0010","Description":"Wireless Headphones","Cost":59.95,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0011","Description":"Stapler","Cost":6.49,"UnitOfMeasure":"EACH","Category":"Office"},{"ProductNumber":"P0012","Description":"Pack of Sticky Notes","Cost":1.99,"UnitOfMeasure":"PACK","Category":"Stationery"},{"ProductNumber":"P0013","Description":"AA Batteries (4 Pack)","Cost":4.25,"UnitOfMeasure":"PACK","Category":"Electronics"},{"ProductNumber":"P0014","Description":"Toaster Oven","Cost":39.95,"UnitOfMeasure":"EACH","Category":"Kitchen"},{"ProductNumber":"P0015","Description":"Water Bottle 1L","Cost":7.89,"UnitOfMeasure":"EACH","Category":"Kitchen"},{"ProductNumber":"P0016","Description":"LED Light Bulbs (6 Pack)","Cost":12.49,"UnitOfMeasure":"PACK","Category":"Home Improvement"},{"ProductNumber":"P0017","Description":"Phone Charger Cable","Cost":8.95,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0018","Description":"Notebook Cooling Pad","Cost":21.99,"UnitOfMeasure":"EACH","Category":"Electronics"},{"ProductNumber":"P0019","Description":"Mechanical Pencil Set (2 Pack)","Cost":4.75,"UnitOfMeasure":"PACK","Category":"Stationery"},{"ProductNumber":"P0020","Description":"Electric Kettle","Cost":24.50,"UnitOfMeasure":"EACH","Category":"Kitchen"}]}

```

Also, for debugging purposes I log the request and the return data in the PASE terminal window. This needs to be removed once you are done testing but is here for demonstration purposes.

```

== Incoming GET request ==
Query string: {"app": "product", "action": "getproductlist"}
Return Data: {"data": [{"ProductNumber": "P0001", "Description": "Wireless Mouse", "Cost": 19.99, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0002", "Description": "Bluetooth Keyboard", "Cost": 29.95, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0003", "Description": "USB-C Charger", "Cost": 14.50, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0004", "Description": "Laptop Backpack", "Cost": 45.00, "UnitOfMeasure": "EACH", "Category": "Accessories"}, {"ProductNumber": "P0005", "Description": "16GB USB Drive", "Cost": 9.99, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0006", "Description": "Coffee Mug", "Cost": 5.25, "UnitOfMeasure": "EACH", "Category": "Kitchen"}, {"ProductNumber": "P0007", "Description": "Notebook - Lined", "Cost": 3.15, "UnitOfMeasure": "EACH", "Category": "Stationery"}, {"ProductNumber": "P0008", "Description": "Ballpoint Pens (Pack of 10)", "Cost": 2.99, "UnitOfMeasure": "PACK", "Category": "Stationery"}, {"ProductNumber": "P0009", "Description": "Desk Lamp", "Cost": 18.75, "UnitOfMeasure": "EACH", "Category": "Office"}, {"ProductNumber": "P0010", "Description": "Wireless Headphones", "Cost": 59.95, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0011", "Description": "Stapler", "Cost": 6.40, "UnitOfMeasure": "EACH", "Category": "Office"}, {"ProductNumber": "P0012", "Description": "Pack of Sticky Notes", "Cost": 1.99, "UnitOfMeasure": "PACK", "Category": "Stationery"}, {"ProductNumber": "P0013", "Description": "AA Batteries (4 Pack)", "Cost": 4.25, "UnitOfMeasure": "PACK", "Category": "Electronics"}, {"ProductNumber": "P0014", "Description": "Toaster Oven", "Cost": 39.95, "UnitOfMeasure": "EACH", "Category": "Kitchen"}, {"ProductNumber": "P0015", "Description": "Water Bottle 1L", "Cost": 7.89, "UnitOfMeasure": "EACH", "Category": "Kitchen"}, {"ProductNumber": "P0016", "Description": "LED Light Bulbs (6 Pack)", "Cost": 12.49, "UnitOfMeasure": "PACK", "Category": "Home Improvement"}, {"ProductNumber": "P0017", "Description": "Phone Charger Cable", "Cost": 8.95, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0018", "Description": "Notebook Cooling Pad", "Cost": 21.99, "UnitOfMeasure": "EACH", "Category": "Electronics"}, {"ProductNumber": "P0019", "Description": "Mechanical Pencil Set (2 Pack)", "Cost": 4.75, "UnitOfMeasure": "PACK", "Category": "Stationery"}, {"ProductNumber": "P0020", "Description": "Electric Kettle", "Cost": 24.50, "UnitOfMeasure": "EACH", "Category": "Kitchen"}]}
== [23/Jul/2025 01:39:12] "GET /api?app=product&action=getproductlist HTTP/1.1" 200 -
Shutdown requested via HTTP
Shutting down server...
== [23/Jul/2025 01:45:25] "GET /api?action=shutdown HTTP/1.1" 200 -
-bash-5.2$

```

If you click the Shutdown button you will see the following message.

```

{"status": "Server shutting down"}

```

In the PASE terminal window you should see something like the following

```

Shutdown requested via HTTP
Shutting down server...
== [23/Jul/2025 01:45:25] "GET /api?action=shutdown HTTP/1.1" 200 -
-bash-5.2$

```

What's Next

Just as our hero lands safely on the far bank, a new enemy emerges from the shadows—a cunning sorcerer cloaked in complexity. His spells? Twisted tables, messy layouts, and raw JSON that's hard to read.

In the next chapter, we'll take the raw data returned from our RPG backend and transform it into a polished web display.

The fight won't be easy, but with the right code and a little courage, even the most chaotic data can be tamed.